OI 板子

1. OI 模板 4
2. __int128 4
    2.1. C 风格 4
    2.2. C++风格 5
3. 小 tips 5
    3.1. 分数比较整数化 5
    3.2. hash 5
4. bit 6
    4.1. qpow(a,x,MOD) 6
    4.2. lowbit(x) 6
    4.3. builtin 系列 6
    4.4. 判断与检测 6
    4.5. 集合操作 6
    4.6. 格雷码 7
5. 二分 8
    5.1. 库函数 8
    5.2. 整数集合上的二分 8
    5.3. 实数域上的二分 9
6. 排序 9
    6.1. 库函数 9
    6.2. 去重 10
    6.3. 离散化 10
    6.4. 中位数 10
    6.5. 逆序数 10
7. 倍增 11
    7.1. 倍增分段 11
    7.2. ST 算法 12
8. 搜索 12
    8.1. BFS 12
    8.2. DFS 12

8.3. ID-DFS 12
8.4. A* 13
8.5. ID-A* 13
9. 双指针 14
9.1. 相向指针 14
9.2. 快慢指针 14
9.3. 滑动窗口 14
9.4. 分离指针 14
10. 进阶数据结构 14
10.1. 并查集 14
10.2. 线段树 15
10.3. 树状数组 17
10.4. 可持久化线段树 17
10.5. 笛卡尔树 19
11. 数论 21
11.1. 质数 21
11.2. gcd 22
11.3. 同余 22
12. 组合数学 26
12.1. 容斥原理 26
12.2. 卡特兰数 26
13. 几何 27
13.1. 凸包 27
13.2. 扫描线 28
14. 动态规划 30
14.1. 背包 dp 30
14.2. 概率 dp 33
15. 图论 34
15.1. 存储结构 34
15.2. 最短路 35
15.3. DAG 37
15.4. 树 37
15.5. 生成树 45
15.6. 二分图 47

15.7. 网络流 48
15.8. 连通性 51
16. 字符串 56
16.1. hash 56
16.2. KMP 模式匹配 56
16.3. Trie 字典树 57
16.4. AC 自动机 58
16.5. SA 59
16.6. SAM 60
17. 随机化 61
17.1. 随机函数 61
17.2. 爬山与退火 62
18. 离线算法 62
18.1. 整体二分 62
18.2. 莫队算法 68

1. OI 模板

```cpp
#include <iostream>
#include <iomanip>
#include <array>
#include <vector>
#include <queue>
#include <stack>
#include <map>
#include <set>
#include <unordered_map>
#include <unordered_set>
#include <bitset>
#include <string>
#include <numeric>
#include <algorithm>
#include <limits>
#include <iterator>
#include <cmath>
#include <cstring>
#include <cstdint>
using namespace std;
#define int long long

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    return 0;
}
```

2. __int128

2.1. C 风格

```cpp
// C 风格
__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x  = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}
void print(__int128 x) {
    if (x < 0) {
        putchar('-');
        x = -x;
    }
    if (x > 9) {
        print(x / 10);
    }
    putchar(x % 10 + '0');
}
```

2.2. C++风格

```cpp
// C++风格
istream& operator>>(istream& is, __int128& a) {
    __int128 x = 0, f = 1;
    char ch;
    is.get(ch);
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        is.get(ch);
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        is.get(ch);
    }
    a = x * f;
    return is;
}
ostream& operator<<(ostream& os, __int128 x) {
    if (x < 0) {
        os << '-';
        x = -x;
    }
    if (x > 9) {
        os << __int128(x / 10);
    }
    os << char(x % 10 + '0');
    return os;
}
```

3. 小 tips

3.1. 分数比较整数化

```cpp
/**
 * @brief 分数比较整数化
 * @note pair<div,mod>来表示一个分数结果
 *       比较运算成立的前提是：除数一致
 */
```

3.2. hash

```cpp
struct Person {
    string name;
    int age;
};
template <>
struct hash<Person> {
    size_t operator()(const Person& p) const noexcept {
        size_t h1 = hash<string>{}(p.name);
        size_t h2 = hash<int>{}(p.age);
        return h1 ^ (h2 + 0x9e3779b97f4a7c15ULL + (h1 << 6) + (h1 >> 2));
    }
};
```

4. bit

4.1. qpow(a,x,MOD)

```
int qpow(int a, int b, int MOD) {
    int ret = 1;
    while (b) {
        if (b & 1) {
            ret = ret * a % MOD;
        }
        a = a * a % MOD;
        b >>= 1;
    }
    return ret;
}
```

4.2. lowbit(x)

```
int lowbit(int x) {
    return x & (-x);
}
```

4.3. builtin 系列

```
/**
 * @brief builtin 系列
 * @note
 *   __builtin_clz(x)      : x 二进制前导 0 个数，x=0 时为未定义
 *   __builtin_ctz(x)      : x 二进制末尾连续 0 的个数，x=0 时未定义
 *   __builtin_ffs(x)      : x 二进制末尾最后一个 1 的位置，编号从 1 开始，x=0 时为 0
 *   __builtin_popcount(x) : x 二进制中 1 的个数
 */
```

4.4. 判断与检测

```
/**
 * @brief 判断与检测
 * @note
 *   x |= (1u << i)       : 第 i 位置 1
 *   x &= ~(1u << i)      : 第 i 位置 0
 *   x ^= (1u << i)       : 第 i 位取反
 *   (x >>i) & 1          : 取第 i 位
 *   lowbit(x) = x & (-x) : 取最低位的 1
 *   x |= (x + 1)         : 把最低位 0 置 1
 *   x &= (x - 1)         : 把最低为 1 置 0
 */
```

4.5. 集合操作

```
/**
 * @brief 集合操作
 * @note
 *   A | B    : 并
 *   A & B    : 交
 *   A & (~B) : 差
```

```
 */
/**
 * @brief 枚举 mask 的所有子集
 */
void fun() {
    for (int s = mask;; s = (s - 1) & mask) {

        // do something

        if (!s) {  // 0 也是子集
            break;
        }
    }
}
```

```
/**
 * @brief 枚举 mask 的所有超集
 */
void fun() {
    for (int s = mask; s < (1 << n); s = (s + 1) | mask) {
        // do something
    }
}
```

```
}
int rev_g(int g) {
    int n = 0;
    for (; g; g >>= 1) {
        n ^= g;
    }
    return n;
}
```

4.6. 格雷码

```
/**
 * @brief 格雷码
 */
int g(int n) {
    return n ^ (n >> 1);
```

5. 二分

5.1. 库函数

```
equal_range();
lower_bound();
upper_bound();
binary_search();
equal_range=[lower_bound,upper_bound)
binary_search=is_valid(equal_range)
```

```
/**
 * @note 在数组有序的情况下统计值域为[L,R]闭区间上的元素个数
 */
int func(int a[], int n, int L, int R) {
    int left_idx  = lower_bound(a, a + n, L) - a;
    int right_idx = upper_bound(a + left_idx, a + n, R) - a;
    return right_idx - left_idx;
}
```

5.2. 整数集合上的二分

```
/**
 * @name lower_bound
 * @brief 返回第一个大于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int lower_bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid 取不到 r
        // 下面三句话配套出现
        int mid = (l + r) >> 1;
        if (a[mid] >= x) {
            r = mid;
        }
        else {
            l = mid + 1;
        }
    }
    // 二分终止条件：l==r
    return l;
}
/**
 * @name
 * @brief 返回最后一个小于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid 取不到 l
        // 下面三句话配套出现
        int mid = (l + r + 1) >> 1;
        if (a[mid] <= x) {
            l = mid;
        }
        else {
            r = mid - 1;
        }
    }
    // 二分终止条件：l==r
    return l;
```

```
}
```

5.3. 实数域上的二分

```
bool calc(double x) {
    return x >= 114514.1919810;
}
double bound(double l, double r) {
    while (l + 1e-5 < r) {
        double mid = (l + r) / 2;
        if (calc(mid)) {
            r = mid;
        }
        else {
            l = mid;
        }
    }
    return l;
}
```

6. 排序

6.1. 库函数

6.1.1. 快速排序

```
sort(iterator begin, iterator end [, __Function cmp]);
```

6.1.2. 稳定排序

```
stable_sort(iterator begin, iterator end [, __Function cmp]);
```

6.1.3. 前`mid-begin`个都有序

```
partial_sort(iterator begin, iterator mid, iterator end [, __Function cmp]);
```

6.1.4. 第`mid-begin`个位置正确

```
nth_element(iterator begin, iterator nth, iterator end [, __Function cmp]);
```

6.1.5. 判断有序

```
is_sorted(iterator begin, iterator end [, __Function cmp]);
```

6.2. 去重

```
sorted.erase(std::unique(sorted.begin(), sorted.end()), sorted.end());
```

6.3. 离散化

```
vector<int> discrete(vector<int> a) {
    if (a.empty()) {
        return {};
    }
    sort(a.begin(), a.end());
    a.erase(std::unique(a.begin(), a.end()), a.end());
    vector<int> r;
    r.push_back(a.front());
    int last = a.front();
    for (auto x : a) {
        if (last != x) {
            r.push_back(x);
            last = x;
        }
    }
    return r;
}
```

6.4. 中位数

中位数到所有点的距离之和最小

6.5. 逆序数

```
int tmp[114514];
int merge(int a[], int l, int r) {  // 左闭右开
    int mid = (l + r) / 2;
    int cnt = 0;
    int i = l, j = mid;
    int k = l;
    while (i < mid && j < r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        }
        else {
            tmp[k++] = a[j++];
            cnt += mid - i;
        }
    }
    while (i < mid) {
        tmp[k++] = a[i++];
    }
    while (j < r) {
        tmp[k++] = a[j++];
    }
    for (k = l; k < r; ++k) {
        a[k] = tmp[k];
    }
    return cnt;
}
int merge_sort(int a[], int l, int r) {
    if (r - l <= 1) {
```

```
        return 0;
    }
    int cnt = 0;
    int mid = (l + r) / 2;
    cnt += merge_sort(a, l, mid);
    cnt += merge_sort(a, mid, r);
    cnt += merge(a, l, r);
    return cnt;
}
```

7. 倍增

7.1. 倍增分段

```
/**
 * @brief 倍增分段
 * @note 左闭右开
 */
void beizeng(vector<int> a) {
    int n = a.size();

    for (int l = 0, r = 0; l < n;) {
        int p = 1;

        while (p) {
            // 左闭右开
            if (r + p <= n && check(l, r + p)) {
                r += p;
                p <<= 1;
            }
            else {
                p >>= 1;
            }
        }

        push_ans(l, r);
        l = r;
    }
}
```

7.2. ST 算法

```
/**
 * @brief ST 算法
 * @details f[i][j]记录[i,i+2^j)内的最大值
 * @details 先进行 O(nlogn)的预处理，之后可以进行 O(1)的在线回答
 */
const int N    = 1e5 + 5;
const int LOGN = 20;
int n;
int a[N];
int f[N][LOGN];
void st_prework() {
    for (int i = 1; i <= n; ++i) {
        f[i][0] = a[i];
    }
    int t = log2(n) + 1;
    for (int j = 1; j < t; ++j) {
        for (int i = 1; i < n - (1 << j); ++i) {
            f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
        }
    }
}
int st_query(int l, int r) {
    int k = log2(r - l + 1);
    return max(f[l][k], f[r - (1 << k) + 1][k]);
}
```

8. 搜索

8.1. BFS

```
void bfs(int x) {
    queue<int> q;
    q.push(x);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if () {
            q.push(v);
        }
    }
}
```

8.2. DFS

```
void dfs(int u) {
    if () {
        return;
    }
    dfs(v);
}
```

8.3. ID-DFS

```cpp
bool iddfs(int u, int d) {
    if (!d) {
        return false;
    }

    iddfs(v, d - 1);
    return true;
}
void solve() {
    for (int deep = 0;; ++deep) {
        if (iddfs(s, deep)) {
            break;
        }
    }
}
```

8.4. A*

```cpp
/**
 * @brief A*
 * @note Dijkstra 版 A*，dis[v]=g[v]+h[v]
 */
struct Edge {
    int v, w;
};
struct Node {  /// Dijkstra 节点
    int dis, u;
    bool operator>(const Node& a) const {
        return dis > a.dis;
    }
};
```

```cpp
vector<Edge> edge[N];
int g[N];
bool vis[N];
priority_queue<Node, vector<Node>, greater<Node>> pq;

void Astar(int s) {
    memset(g, 0x3f, sizeof(g));

    g[s] = 0;

    pq.push({h[0], s});

    while (!pq.empty()) {
        int u = pq.top().u;
        pq.pop();
        if (vis[u]) {
            continue;
        }
        vis[u] = 1;
        for (auto ed : edge[u]) {
            int v = ed.v, w = ed.w;
            if (g[v] > g[u] + w) {
                g[v] = g[u] + w;
                pq.push({g[v] + h[v], v});
            }
        }
    }
}
```

8.5. ID-A*

9. 双指针

9.1. 相向指针

8.1.1. 接雨水问题

8.1.2. 最大矩形

9.2. 快慢指针

8.2.1. 环问题

9.3. 滑动窗口

8.3.1. 最小覆盖字串

8.3.2. 无重复字符的最长字串

9.4. 分离指针

8.4.1. 归并排序

10. 进阶数据结构

10.1. 并查集

```
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    size_t find_root(size_t x) {
        return parent[x] == x
                    ? x
                    : parent[x] = find_root(parent[x]);
    }
    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};
```

10.2. 线段树

```
/**
 * @note 只有满足结合律的运算才可使用
 * @note 此模板是加法版的 SegTree
 */

// 左闭右闭
struct Interval {
    int left, right;  // 所管理 a 的区间
    int root;         // 对应 d 的节点

    int mid(void) const {
        return left + (right - left) / 2;
    }
    int len(void) const {
        return right - left + 1;
    }

    int left_root(void) const {
        return root * 2;
    }
    int right_root(void) const {
        return root * 2 + 1;
    }

    int left_len(void) const {
        return mid() - left + 1;
    }
    int right_len(void) const {
        return right - mid();
    }

    Interval left_interval(void) const {
        return {left, mid(), root * 2};
    }
    Interval right_interval(void) const {
        return {mid() + 1, right, root * 2 + 1};
    }

    bool is_in(int find_left, int find_right) const {
        return find_left <= left && find_right >= right;
    }
};
struct SegTree {
    int a[N];  // 原始数据

    int d[4 * N];     // 节点
    int lazy[4 * N];  // 懒标记

    SegTree() {
        fill(d, d + 4 * N, 0);
        fill(lazy, lazy + 4 * N, 0);
    }
    void push_up(Interval interval) {
        if (interval.left != interval.right) {
            d[interval.root] = d[interval.left_root()] + d[interval.right_root()];
        }
    }
    // 由 a 数组建树
    void build(Interval interval) {
        if (interval.left == interval.right) {
            d[interval.root] = a[interval.left];
```

```
            return;
        }
        build(interval.left_interval());
        build(interval.right_interval());
        push_up(interval);
    }
    // 分发懒标记
    void push_down(Interval interval) {
        // 判断：有懒标记且不是叶子节点
        if (lazy[interval.root] && interval.left < interval.right) {
            // 应用懒标记
            d[interval.left_root()]  = d[interval.left_root()] + lazy[interval.root] *
interval.left_len();
            d[interval.right_root()] = d[interval.right_root()] + lazy[interval.root] *
interval.right_len();
            // 下发懒标记
            lazy[interval.left_root()]  = lazy[interval.left_root()] +
lazy[interval.root];
            lazy[interval.right_root()] = lazy[interval.right_root()] +
lazy[interval.root];
            // 清空懒标记
            lazy[interval.root] = 0;
        }
    }

    // 区间查找(find_left==find_right 时为单点查找)
    int query(int find_left, int find_right, Interval interval) {
        if (interval.is_in(find_left, find_right)) {
            return d[interval.root];
        }
        push_down(interval);

        int mid = interval.mid();
        int sum = 0;  // 单位元
        if (find_left <= mid) {
            sum += query(find_left, find_right, interval.left_interval());
        }
        if (find_right > mid) {
            sum += query(find_left, find_right, interval.right_interval());
        }
        return sum;
    }

    // 区间更新(update_left==update_right 时为单点更新)
    void update(int update_left, int update_right, int delta_val, Interval interval) {
        if (interval.is_in(update_left, update_right)) {
            d[interval.root] += delta_val * interval.len();  // 更新节点
            lazy[interval.root] += delta_val;                // 更新懒标记
            return;
        }
        push_down(interval);

        int mid = interval.mid();
        if (update_left <= mid) {
            update(update_left, update_right, delta_val, interval.left_interval());
        }
        if (update_right > mid) {
            update(update_left, update_right, delta_val, interval.right_interval());
        }
        push_up(interval);
    }
};
SegTree seg;
signed main() {
    Interval init_interval{1, n, 1};
```

```
    for (int i = 1; i <= n; ++i) {
        cin >> seg.a[i];
    }
    seg.build(init_interval);
    seg.update(update_left, update_right, 114514, init_interval);
    cout << seg.query(find_left, find_right, init_interval);
    return 0;
}
```

10.3. 树状数组

```
/**
 * @note 只有满足结合律和差分性质的运算才可使用
 * @note 支持单点修改和区间查询
 * @note i 父亲: i+lowbit(i)
 */
int lowbit(int x) {
    return x & (-x);
}
int a[N];
int n;
int c[N];  // 树状数组节点，c[i] 管理 (i - lowbit(i), i] 这段区间

void build() {
    for (int i = 1; i <= n; ++i) {
        c[i] += a[i];
        int j = i + lowbit(i);
        if (j <= n) {
            c[j] += c[i];
        }
    }
}
```

```
// 单点更新
void add(int pos, int val) {
    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {
    return presum(right) - presum(left - 1);
}
/**
 * @note 若要求区间修改单点查询，则改为差分数组
 * @note 若要求区间修改区间查询，则维护两个数组 d1[],d2[]
 *       d[i]=a[i]-a[i-1]
 *       sum a[i]=sum d[i]*(n-i+1)=sum d[i]*(n+1) + sum d[i]*i
 *       因此 d1[i]=d[i],d2[i]=d[i]*i
 */
```

10.4. 可持久化线段树

```
const int N       = 1e5 + 5;
const int MAXNODE = N * 24;

int lr[MAXNODE];  // lr[node]
```

```
int rr[MAXNODE];  // rr[node]

struct Interval {
    int left, right;  // 所管理 a 的区间
    int node;         // 对应 d 的节点

    inline int mid(void) const {
        return left + (right - left) / 2;
    }
    inline int len(void) const {
        return right - left + 1;
    }

    inline int left_len(void) const {
        return mid() - left + 1;
    }
    inline int right_len(void) const {
        return right - mid();
    }

    inline int left_node(void) const {
        return lr[node];
    }
    inline int right_node(void) const {
        return rr[node];
    }

    inline Interval left_interval(void) const {
        return {left, mid(), left_node()};
    }
    inline Interval right_interval(void) const {
        return {mid() + 1, right, right_node()};
    }

    inline bool is_leaf(void) const {
        return left == right;
    }
    inline bool is_in(int find_left, int find_right) const {
        return find_left <= left && find_right >= right;
    }
};

int a[N];
int d[MAXNODE];     // d[node]
int lazy[MAXNODE];  // lazy[node]
int node_cnt;

int root[N];  // root[version]
int version_cnt;

inline int new_node() {
    return ++node_cnt;
}
int clone_new_node(int from) {
    int node   = new_node();
    lr[node]   = lr[from];
    rr[node]   = rr[from];
    d[node]    = d[from];
    lazy[node] = lazy[from];
    return node;
}
void push_up(Interval interval) {
    if (!interval.is_leaf()) {
        d[interval.node] = d[interval.left_node()] + d[interval.right_node()] +
```

```
lazy[interval.node] * interval.len();
    }
}
int build(Interval interval) {  // interval.node 无需填
    interval.node = new_node();
    if (interval.is_leaf()) {
        d[interval.node] = a[interval.left];
        return interval.node;
    }
    lr[interval.node] = build(interval.left_interval());
    rr[interval.node] = build(interval.right_interval());
    push_up(interval);
    return interval.node;
}

int query(Interval interval, int find_left, int find_right) {  // interval.node 填
root[version]
    if (interval.is_in(find_left, find_right)) {
        return d[interval.node];
    }

    int mid = interval.mid();
    int sum = lazy[interval.node] * (min(find_right, interval.right) - max(find_left,
interval.left) + 1);
    if (find_left <= mid) {
        sum += query(interval.left_interval(), find_left, find_right);
    }
    if (find_right > mid) {
        sum += query(interval.right_interval(), find_left, find_right);
    }
    return sum;
}
int update(Interval interval, int find_left, int find_right, int delta_val) {  //
```

```
interval.node 无需填
    interval.node = clone_new_node(interval.node);
    if (interval.is_in(find_left, find_right)) {
        d[interval.node] += delta_val * interval.len();
        lazy[interval.node] += delta_val;
        return interval.node;
    }

    int mid = interval.mid();
    if (find_left <= mid) {
        lr[interval.node] = update(interval.left_interval(), find_left, find_right,
delta_val);
    }
    if (find_right > mid) {
        rr[interval.node] = update(interval.right_interval(), find_left, find_right,
delta_val);
    }
    push_up(interval);
    return interval.node;
}

signed main() {
    root[version_cnt++] = build({1, n, 0});
    root[version_cnt++] = update({1, n, root[some_version]}, 1145, 1919, 14810);
    ans                 = query({1, n, root[some_version]}, 1145, 1919);
    return 0;
}
```

10.5. 笛卡尔树

```
/**
 * @brief 笛卡尔树
```

```
 * @note
 *   1. 笛卡尔树是一种二叉树，每个节点由(k,w)构成
 *   2. k 满足 BST 的性质：中序遍历即为 k 的顺序
 *      w 满足堆的性质：最值在根处
 *   3. 若 k 和 w 分别互不相同，则笛卡尔树唯一
 *   4. 一般地，k=idx，w=a[idx]
 */
int a[N];  // k=idx,w=a[idx]
int n;
struct Node {
    int left, right;
    int parent;
};
struct CartesianTree {
    vector<Node> node;
    int root;
    CartesianTree() {
        node.assign(n + 1, {0, 0, 0});
        root = 0;
    }
    int build() {  // 单调栈维护右子树
        vector<int> stk(n + 1, 0);
        int top = 0;
        for (int i = 1; i <= n; ++i) {
            int k = top;
            while (k && a[stk[k]] > a[i]) {
                --k;
            }
            if (k) {
                node[stk[k]].right = i;
                node[i].parent     = stk[k];
            }
            else {
                root = i;
            }
            if (k < top) {
                node[i].left            = stk[k + 1];
                node[stk[k + 1]].parent = i;
            }
            stk[++k] = i;
            top      = k;
        }
        return root;
    }
};
```

11. 数论

11.1. 质数

11.1.1. 质数的判定

```
/**
 * @brief 试除法
 */
bool is_prime(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```

11.1.2. 质数筛法

11.1.2.1. 埃氏筛

```
/**
 * @brief 埃式筛
 * @details O(n loglogn)
 */
const int N = 10000;
bitset<N> v;
vector<int> primes;
void get_primes_1(int n) {
    v.reset();

    for (int i = 2; i <= n; ++i) {
        if (v[i]) {
            continue;
        }
        primes.push_back(i);
        for (int j = i * i; j <= n; j += i) {
            v[j] = true;
        }
    }
}
```

11.1.2.2. 线性筛

```
/**
 * @brief 线性筛
 * @details O(n)
 */
int u[N];            // 记录最小的质因子
vector<int> primes;  // 质数列表
void get_primes_2(int n) {
    memset(u, 0, sizeof(u));
    for (int i = 2; i <= n; ++i) {
        if (u[i] == 0) {
```

```
            u[i] = i;
            primes.push_back(i);
        }
        for (auto x : primes) {
            if (x > u[i] || x > n / i) {
                break;
            }
            u[i * x] = x;
        }
    }
}
```

11.1.3. 质因数分解

```
struct Prime {
    int p, c;
};
vector<Prime> p;
void divide(int n) {
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int cnt = 0;
            while (n % i == 0) {
                n /= i;
                cnt++;
            }
            p.push_back({i, cnt});
        }
    }
    if (n > 1) {
        p.push_back({n, 1});
    }
}
```

```
    }
}
```

11.2. gcd

```
/**
 * @name gcd
 */
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}
/**
 * @name lcm
 */
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

11.3. 同余

11.3.1. 费马小定理

```
/**
 * @brief 费马小定理
 * @note
 *   1. p 是素数，a^p=a(mod p)
 *   2. p 是素数，p 不整除 a，a^{p-1}=1(mod p)
 */
```

11.3.2. 欧拉函数

```
/**
 * @brief 欧拉函数 phi(n)
 * @note
 *   1. 小于等于 n 且与 n 互素的数的个数
 *   2. 若 gcd(a,b)=1，则 phi(ab)=phi(a)phi(b)
 *     - n = sum_{d|n} phi(d)
 *   3. 若 p 为质数，则 phi(p^k)=p^k-p^(k-1)
 *     - n = n * mul (1-1/p_i)，p_i 为 n 的质因数
 */
// 单点求解：在质因数分解的同时求即可
int euler_phi(int n) {
    int ans = n;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            ans = ans / i * (i - 1);
            while (n % i == 0) {
                n /= i;
            }
        }
    }
    if (n > 1) {
        ans = ans / n * (n - 1);
    }
    return ans;
}
// 批量预处理：线性筛
int u[N];           // 记录最小的质因子
vector<int> primes;  // 质数列表
int phi[N];
void euler_phi_pre(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (u[i] == 0) {
            u[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (auto x : primes) {
            if (x > n / i) {
                break;
            }
            u[i * x] = x;
            if (i % x == 0) {
                phi[i * x] = phi[i] * x;
                break;
            }
            phi[i * x] = phi[i] * phi[x];
        }
    }
}
```

11.3.3. 欧拉定理

```
/**
 * @brief 欧拉定理
 * @note
 *   1. a^phi(m)=1(mod m)
 *   2. m 为素数 p 时，phi(p)=p-1
 *   3. 扩展欧拉定理(mod m 下)
 *      a^k=
 *          a^(k mod phi(m))         ,gcd(a,m)=1
 *          a^k                      ,gcd(a,m)!=1,k<phi(m)
```

```
 *          a^(k mod phi(m) + phi(m)),gcd(a,m)!=1,k>=phi(m)
 *   4. a^k(mod m)随着 k 增大会出现纯循环和混循环
 */
```

11.3.4. 模逆元

```
/**
 * @brief 单个模逆元
 */
// 扩展欧几里得算法，适用于所有逆元存在 O(log min{a,MOD})
void ex_gcd(int a, int b, int& x, int& y) {
    if (!b) {
        x = 1;
        y = 0;
        return;
    }
    ex_gcd(b, a % b, y, x);
    y -= a / b * x;
}
int inverse1(int a, int MOD) {
    int x, y;
    ex_gcd(a, MOD, x, y);
    return (x % MOD + MOD) % MOD;
}
// 快速幂算法，适用于 MOD 是素数的情况，O(log MOD)
int inverse2(int a, int MOD) {
    return qpow(a, MOD - 2, MOD);
}
```

```
/**
 * @brief 多个模逆元
 * @details 模的逆元就是逆元的乘积，需要预先处理乘积
 */
int a[N];
int S[N];
int ans[N];
void solve() {
    S[0] = 1;
    for (int i = 1; i <= n; ++i) {
        S[i] = S[i - 1] * a[i] % MOD;
    }
    int S_inv = inverse(S[n], MOD);
    for (int i = n; i >= 1; --i) {
        ans[i] = S_inv * S[i - 1] % MOD;
        S_inv  = S_inv * a[i] % MOD;
    }
}
```

11.3.5. 阶乘取模

```
/**
 * @brief 阶乘取模
 * @note
 *   1. Wilson 定理：p 是素数等价于(m-1)! mod m=-1
 *      即(m!)_m=-1(mod m)
 *   2. 令 P=\Pi_{k=1...m,(k,m)=1}=-1(mod m)
 *      上式取-1 当且仅当 m 存的原根存在
 *      原根存在：
 *      m=2,4,p^a,2p^a(p 为奇素数，a 为正整数)
 *   3. 对于素数 p 和正整数 n，(mod p 下)
 *      (n!)_p=(-1)^floor(n/p)
 *            *(n mod p)!
 *            *(floor(n/p)!)_p
 *      递归深度为 O(lnn/lnp)，复杂度 O(p lnn/lnp)
 *      若 p 很小可以直接对小于 p 的 n 预处理出来(O(p))
```

```
 *      则总复杂度 O(p+lnn/lnp)
 */
int f[N];
void init(int p) {
    f[0] = 1;
    for (int i = 1; i < p; ++i) {
        f[i] = f[i - 1] * i % p;
    }
}
int fastmod(int n, int p) {  //(n!)_p mod p
    int res = 1;
    while (n > 1) {
        if ((n / p) & 1) {
            res = p - res;
        }
        res = res * f[n % p] % p;
        n /= p;
    }
    return res;
}
```

11.3.6. 中国剩余定理

```
/**
 * @brief 中国剩余定理
 * @note
 *   方程组:
 *     x = a[i] (mod r[i]) i=1,2,...,k
 *     其中，r[i]两两互质
 *   过程:
 *     1. n = mul r[i]
 *     2. m[i] = n / r[i]
 *        inv_m[i] = ... (mod r[i])
 *        c[i] = m[i] * inv_m[i]
 *     3. x = sum a[i]*c[i] (mod n)
 */
void ex_gcd(int a, int b, int& x, int& y) {
    if (!b) {
        x = 1;
        y = 0;
        return;
    }
    ex_gcd(b, a % b, y, x);
    y -= a / b * x;
}
int inverse(int a, int MOD) {
    int x, y;
    ex_gcd(a, MOD, x, y);
    return (x % MOD + MOD) % MOD;
}
int CRT(int k, int a[N], int r[N]) {
    int n   = 1;
    int ans = 0;
    for (int i = 1; i <= k; ++i) {
        n *= r[i];
    }
    for (int i = 1; i <= k; ++i) {
        int m     = n / r[i];
        int inv_m = inverse(m, r[i]);
        ans       = (ans + a[i] * m * inv_m % n) % n;
    }
    return (ans % n + n) % n;
}
```

11.3.7. 裴蜀定理

```
/**
 * @brief 裴蜀定理
 * @note
 *   1. 定理：若 a,b 不全为 0，则
 *     ax+by=k 有正整数解，当且仅当且 gcd(a,b)|k
 *   2. ax+by=1 有正整数解，当且仅当且 a,b 互素
 */
```

12. 组合数学

12.1. 容斥原理

```
/**
 * @def
 *   方便书写，定义 A[i]与 a[i]相斥，全集为 U
 * @note
 *   1. A1 元积-A2 元积+...+(-1)^n An 元积=U-an 元积
 *   2. 当进行容斥的情况数量过多时，可以对全集 U 进行缩水操作，再在新的全集讨论情况：
 *      U-(A+B)=(U-B)-A
 *   3. P[i]=a[1]a[2]...a[n-1]A[n]可以方便进行 dp
 *      a[1]a[2]...a[n-1]A[n]=A[n]-sum(a[1]a[2]...a[i-1]A[i]A[n])
 */
```

12.2. 卡特兰数

```
/**
 * @brief Catalan 数
 * @note
 *   1. 表达式
 *     a. C[n]=C(2n,n)/(n+1)
 *     b. C[n]=(2n)!/(n!(n+1)!)
 *     c. C[n]=C(2n,n)-C(2n,n+1)
 *     d. C[n]=C[n-1] (4n-2)/(n+1),C[0]=C[1]=1
 *     e. C[n]=sum^(n-1)_0{C[i]C[n-1-i]}
 *   2. 表征意义
 *     a. 给定正负一序列{a[i]},前缀和 S[i]:=sum(a,1,i)
```

```
*         需保证任意 i,S[i]>=0
*         超额量 T:=S[i]<0 且 a[i]=-1 的数量
*         可以证明，T 属于[0,n]，且每种情况的数列数量都相同
*     c. 从(0,0)走到(n,n)，只能向右或向上，不允许在 y=x 直线上方
*         可通过 y=x+1 直线对称过去，每一种非法情况都对应(0,0)走到(n-1,n+1)的路径
*         总=C(2n,n)，非法=C(2n,n+1)
*     de. n+2 边形的三角剖分方案数
*     de. 含有 n 个节点的形态不同的二叉树数目
*/
```

13. 几何

13.1. 凸包

```
/**
 * @brief Andrew 算法
 * @details O(n log n)
 */
struct Point {
    int x, y;
    Point operator+(const Point& _) const {
        return Point{x + _.x, y + _.y};
    }
    Point operator-(void) const {
        return Point{-x, -y};
    }
    Point operator-(const Point& _) const {
        return *this + (-_);
    }
    int operator*(const Point& _) const {
        return x * _.y - _.x * y;
    }
};

vector<int> s;  // 单调栈，记录下标
vector<Point> p;
bool used[N];
void Andrew() {
    // 确定左下角和右下角，一定是凸包顶点
    sort(p.begin(), p.end(), [&](const Point& a, Point& b) -> bool {
```

```
        if (a.x != b.x) {
            return a.x < b.x;
        }
        return a.y < b.y;
    });

    // 下凸壳
    s.push_back(0);
    for (int i = 1; i < s.size(); ++i) {
        while (s.size() >= 2 && (p[s[s.size() - 1]] - p[s[s.size() - 2]]) * (p[i] - p[s[s.size()
- 1]]) <= 0) {
            used[s.back()] = 0;
            s.pop_back();
        }
        used[i] = 1;
        s.push_back(i);
    }

    // 上凸壳
    int bottom_size = s.size();
    for (int i = s.size() - 1; i >= 2; --i) {
        if (!used[i]) {
            while (s.size() > bottom_size &&
                   (p[s[s.size() - 1]] - p[s[s.size() - 2]]) * (p[i] - p[s[s.size() - 1]])
<= 0) {
                used[s.back()] = 0;
                s.pop_back();
            }
            used[i] = 1;
            s.push_back(i);
        }
    }
}
```

13.2. 扫描线

```
/**
 * @brief P5490 离散化+存区间的线段树+扫描线
 * @details 给 n 个边平行于坐标轴的矩形，求它们并集的总面积
 * @details 1 <= n <= 1e5 0 <= x1 < x2 <= 1e9 0 <= y1 < y2 <= 1e9
 */
#include...
using namespace std;
#define int long long
const int N = 2e5 + 5;
struct rec {
    int y1, y2;
    int x1;
    int value;
    bool operator<(const rec& _) const {
        if (x1 != _.x1) {
            return x1 < _.x1;
        }
        return y1 < _.y1;
    }
};
vector<rec> r;
vector<int> y;  // 对 y 值进行离散化

struct Interval {
    int left;
    int right;
    int root;

    int mid(void) const {
```

```
        return left + ((right - left) >> 1);
    }
    int len(void) const {
        return right - left + 1;
    }
    int left_root(void) const {
        return root * 2;
    }
    int right_root(void) const {
        return root * 2 + 1;
    }

    int left_len(void) const {
        return mid() - left + 1;
    }
    int right_len(void) const {
        return right - mid();
    }
    Interval left_interval(void) const {
        return {left, mid(), left_root()};
    }
    Interval right_interval(void) const {
        return {mid() + 1, right, right_root()};
    }
    bool is_in(int find_left, int find_right) const {
        return find_left <= left && find_right >= right;
    }
};

int len[N * 4];  // root 对应的区间被覆盖的总长度
int cnt[N * 4];  // root 对应的区间被覆盖的次数
```

```
void push_up(Interval interval) {
    if (cnt[interval.root] > 0) {
        len[interval.root] = y[interval.right + 1] - y[interval.left];
    }
    else if (interval.left == interval.right) {
        len[interval.root] = 0;
    }
    else {
        len[interval.root] = len[interval.left_root()] + len[interval.right_root()];
    }
}
void update(Interval interval, int find_left, int find_right, int k) {
    if (interval.is_in(find_left, find_right)) {
        cnt[interval.root] += k;
        push_up(interval);
        return;
    }
    int mid = interval.mid();
    if (find_left <= mid) {
        update(interval.left_interval(), find_left, find_right, k);
    }
    if (find_right > mid) {
        update(interval.right_interval(), find_left, find_right, k);
    }
    push_up(interval);
}
int query() {
    return len[1];
}

signed main() {
    ios::sync_with_stdio(false);
```

```
    cin.tie(0);

    int n;
    cin >> n;

    for (int i = 0; i < n; ++i) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        r.push_back({y1, y2, x1, 1});
        r.push_back({y1, y2, x2, -1});
        y.push_back(y1);
        y.push_back(y2);
    }

    sort(r.begin(), r.end());

    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());

    // build(Interval{0, (int)y.size() - 1, 1});

    int ans = 0;
    for (int j = 1; j < 2 * n; ++j) {
        int i  = j - 1;
        int y1 = lower_bound(y.begin(), y.end(), r[i].y1) - y.begin();
        int y2 = lower_bound(y.begin(), y.end(), r[i].y2) - y.begin();
        update(Interval{0, (int)y.size() - 1, 1}, y1, y2 - 1, r[i].value);
        ans += query() * (r[i + 1].x1 - r[i].x1);
    }
    cout << ans;
    return 0;
}
```

14. 动态规划

14.1. 背包 dp

14.1.1. 01 背包

```
struct item {
    int w, v;
};
item a[N];
// int dp01[N][W];
int dp01[W];
void beibao01() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }

    // 法一：二维数组
    // for (int i = 1; i <= n; ++i) {
    //     for (int j = 0; j < a[i].w; ++j) {
    //         dp01[i][j] = dp01[i - 1][j];
    //     }
    //     for (int j = a[i].w; j <= w; ++j) {
    //         dp01[i][j] = max(dp01[i - 1][j], dp01[i - 1][j - a[i].w] + a[i].v);
    //     }
    // }
    // cout << dp01[n][w];
```

```
    // 法二：一维数组（注意遍历方向）
    for (int i = 1; i <= n; ++i) {
        for (int j = w; j >= a[i].w; --j) {
            dp01[j] = max(dp01[j], dp01[j - a[i].w] + a[i].v);
        }
    }
    cout << dp01[w];
}
```

14.1.2. 多重背包

```
struct item {
    int w, v;
    int c;
};
item a[N];

int dpdc[W];
void beibaodc() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v >> a[i].c;
    }

    // 法一：当成 01 背包，物品复制 c 份
    // for (int i = 1; i <= n; ++i) {
    //     // 小剪枝：如果足够大，可以直接当成完全背包
    //     // begin
    //     if (a[i].c * a[i].w >= w) {
    //         for (int j = a[i].w; j <= w; ++j) {
    //             dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
    //         }
    //         continue;
    //     }
    //     // end
    //     while (a[i].c--) {
    //         for (int j = w; j >= a[i].w; --j) {
    //             dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
    //         }
    //     }
    // }

    // 法二：二进制优化
    for (int i = 1; i <= n; ++i) {
        // 小剪枝：如果足够大，可以直接当成完全背包
        // begin
        if (a[i].c * a[i].w >= w) {
            for (int j = a[i].w; j <= w; ++j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
            }
            continue;
        }
        // end

        // 二进制优化
        int leiji = 0;
        for (int k = 1; leiji + k <= a[i].c; leiji += k, k <<= 1) {
            for (int j = w; j >= a[i].w * k; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * k] + a[i].v * k);
            };
        }
```

```
        int r = a[i].c - leiji;
        if (r) {
            for (int j = w; j >= a[i].w * r; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * r] + a[i].v * r);
            }
        }
    }
    cout << dpdc[w];
}
```

14.1.3. 完全背包

```
struct item {
    int w, v;
    int c;
};
item a[N];

int dpdc[W];
void beibaodc() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v >> a[i].c;
    }

    // 法一：当成 01 背包，物品复制 c 份
    // for (int i = 1; i <= n; ++i) {
    //     // 小剪枝：如果足够大，可以直接当成完全背包
    //     // begin
    //     if (a[i].c * a[i].w >= w) {
    //         for (int j = a[i].w; j <= w; ++j) {
    //             dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
    //         }
    //         continue;
    //     }
    //     // end
    //     while (a[i].c--) {
    //         for (int j = w; j >= a[i].w; --j) {
    //             dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
    //         }
    //     }
    // }

    // 法二：二进制优化
    for (int i = 1; i <= n; ++i) {
        // 小剪枝：如果足够大，可以直接当成完全背包
        // begin
        if (a[i].c * a[i].w >= w) {
            for (int j = a[i].w; j <= w; ++j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
            }
            continue;
        }
        // end

        // 二进制优化
        int leiji = 0;
        for (int k = 1; leiji + k <= a[i].c; leiji += k, k <<= 1) {
            for (int j = w; j >= a[i].w * k; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * k] + a[i].v * k);
            };
        }
```

```
        int r = a[i].c - leiji;
        if (r) {
            for (int j = w; j >= a[i].w * r; --j) {
                dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * r] + a[i].v * r);
            }
        }
    }
    cout << dpdc[w];
}

/**
 * @brief 完全背包
 */

struct item {
    int w, v;
};
item a[N];

int dpwq[W];
void beibaowq() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = a[i].w; j <= w; ++j) {
            dpwq[j] = max(dpwq[j], dpwq[j - a[i].w] + a[i].v);
        }
    }
    cout << dpwq[w];
}
```

14.1.4. 分组背包

14.2. 概率 dp

```
/**
 * @brief 期望 dp
 * @note
 *   1. 条件期望公式:   E[A] = \sum P[A|B_i]*E[A|B_i]
 *      全概率期望公式: E[v] = S[v] / P[v]
 *                     S[u] = P[u] * E[u]
 *                     S[v] = \sum_u p[u->v]*S[u] + \sum_u P[u]*p[u->v]*w[u->v]
 *   2. 应保证 dp 的无后效性，必须由已知推未知，决定了正序还是倒序遍历，决定了是否排序
 *      正序/倒序的判断方法：是否影响后续
 *                          "a 状态有 p_i 概率变成 b_i"->倒序
 *   3. 从叶子往根推比较好推
 */
```

15. 图论

15.1. 存储结构

15.1.1. 链式前向星

```
int n;
struct Edge {
    int next;    // 下一个坐标
    int to;      // 指向的节点
    int weight;  // 边权重
};
vector<Edge> edge;
vector<int> head;  // 头节点的初始坐标
void init_edge(int n) {
    edge.clear();
    head.assign(n + 1, -1);
}
void add_edge(int u, int v, int w) {
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;
}
bool find_edge(int u, int v) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        if (edge[i].to == v) {
            return true;
        }
    }
    return false;
}
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;

        dfs(v);
    }
}
```

15.1.2. 邻接表

```
struct Edge {
    int v, w;
};
int n;
vector<vector<int>> edge;
void init_edge(int n) {
    edge.assign(n + 1, vector<Edge>());
}
void add_edge(int u, int v, int w) {
    edge[u].push_back({v, w});
};
bool find_edge(int u, int v) {
    for (auto [_v, w] : edge[u]) {
        if (v == _v) {
```

```
            return true;
        }
    }
    return false;
}
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (auto v : edge[u]) {
        dfs(v);
    }
}
```

15.1.3. 邻接矩阵

```
vector<vector<int>> graph;
int n;
void init_edge(int n) {
    graph.assign(n + 1, vector<int>(n + 1, 0));
}
void add_edge(int u, int v, int w) {
    graph[u][v] = w;
};
bool find_edge(int u, int v) {
    return graph[u][v] != 0;
}
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (int v = 1; v <= n; ++v) {
        if (graph[u][v]) {
            dfs(v);
        }
    }
}
```

15.2. 最短路

15.2.1. Dijkstra

```
/**
 * @brief Dijkstra
 * @details time O(m log n)
 * @details 只允许非负权图
 */
const int N   = 1e3;
const int INF = 0x3f3f3f3f;
int n;
struct Edge {
    int v, w;
};
struct Node {
    int u;
    int dis;
    bool operator>(const Node& a) const {
        return dis > a.dis;
```

```
    }
};
vector<Edge> edge[N];
int dis[N];
bool vis[N];
priority_queue<Node, vector<Node>, greater<Node>> pq;

void dijkstra(int s) {
    memset(dis, 0x3f, (n + 1) * sizeof(dis[0]));


    dis[s] = 0;
    pq.push({s, 0});
    while (!pq.empty()) {
        int u = pq.top().u;
        pq.pop();
        if (vis[u]) {
            continue;
        }
        vis[u] = true;
        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                pq.push({v, dis[v]});
            }
        }
    }
}
```

15.2.2. SPFA

```
/**
 * @name SPFA
 * @details time<=O(nm) space O(n)
 * @details 适用于正负边权的图，可以判断无最短路的情况
 */
const int N   = 1e3;
const int INF = 0x3f3f3f3f;
struct Edge {
    int v, w;
};
int n;
vector<Edge> edge[N];
int dis[N];
int cnt[N];
bool vis[N];

queue<int> q;
bool SPFA(int s) {
    memset(dis, 0x3f, (n + 1) * (sizeof(dis[0])));

    dis[s] = 0;
    vis[s] = true;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        vis[u] = false;

        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {
```

```
                dis[v] = dis[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) {
                    return true;
                }
                if (!vis[v]) {
                    q.push(v);
                    vis[v] = true;
                }
            }
        }
    }
    return false;
}
```

15.3. DAG

15.3.1. 拓扑排序

```
/**
 * @brief 拓扑排序
 * @note  能拓扑排序与 DAG 互为充分必要
 * @return 是否为 DAG
 */
// Kahn 算法
bool topological_sort() {
    queue<int> s;
    vector<int> ans;
    for (int i = 1; i <= n; ++i) {
        if (!in_degree[i]) {
            s.push(i);
        }
    }
    while (!s.empty()) {
        int u = s.front();
        s.pop();
        ans.push_back(u);
        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            --in_degree[v];
            if (!in_degree[v]) {
                s.push(v);
            }
        }
    }
    if (ans.size() == n) {
        for (auto i : ans) {
            cout << i << ' ';
        }
        return true;
    }
    return false;
}
```

15.4. 树

15.4.1. 深度、2^k 辈父节点

```
/**
 * @brief depth 和 father
```

```
 * @param d[x]:表示 x 深度
 * @param f[x][k]:表示 x 的 2^k 辈祖先
 * @details time O(n)
 */

int d[N];
int f[N][LOGN];

void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t   = (int)(log(n) / log(2)) + 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);

            d[v] = d[u] + 1;

            f[v][0] = u;
            for (int j = 1; j <= t; ++j) {
                f[v][j] = f[f[v][j - 1]][j - 1];
            }
        }
    }
}
```

15.4.2. 树的中心

```
/**
 * @brief  树的中心
 * @property 1.在直径上
 * @property 2.中心不唯一，但最多有 2 个，且两个中心相邻
 * @property 3.所有点到最远点的路径一定交汇于树的中心
 * @property 4.当中心为根节点时，到达直径两端的两条链一定分别是最长链和次长链
 * @property 5.树的中心到其他节点距离不超过直径的一半
 * @details 求法：寻找一个点 x，使其作为根节点时的最长链长度最短
 * @details time O(n)
 */
int d1[N];  // 以 1 为节点时 x 子树的最长链
int d2[N];  // 以 1 为节点时 x 子树的次长链
int up[N];  // x 子树外的最长链，必然经过 x 的父节点
// 如果找到点 x 使得 max(d1[x],up[x])最小，则 x 即为树的中心
void dfs_down(int u, int father) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        dfs_down(v, u);

        if (d1[v] + w > d1[u]) {
            d2[u] = d1[u];
```

```
            d1[u] = d1[v] + w;
        }
        else if (d1[v] + w > d2[u]) {
            d2[u] = d1[v] + w;
        }
    }
}
void dfs_up(int u, int father) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        up[v] = up[u] + w;
        if (d1[v] + w != d1[u]) {
            up[v] = max(up[v], d1[u] + w);
        }
        else {
            up[v] = max(up[v], d2[u] + w);
        }
        dfs_up(v, u);
    }
}
int get_center() {
    dfs_down(1, 0);
    dfs_up(1, 0);
    int minlen = INT_MAX;
    int ans    = 0;
    int ans2   = 0;
    for (int i = 1; i <= n; ++i) {
        // 其中一个中心
        if (max(d1[i], up[i]) < minlen) {
            minlen = max(d1[i], up[i]);
            ans    = i;
            ans2   = 0;
        }
        // 另一个中心
        else if (max(d1[i], up[i]) == minlen) {
            ans2 = i;
        }
    }
    return ans;
}
```

15.4.3. LCA

```
/**
 * @brief 树的直径
 * @details 树形 dp
 * @param d[x]: 以 x 节点为根节点，向下延申可形成的最长距离
 * @param ans: 经过 x 节点形成的最长直径
 */
int d[N];
int ans;
void dp(int u, int fa) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;

        if (v == fa) {
            continue;
        }
```

```
        dp(v, u);

        // 这里的 d[u]为 1-(i-1)内的最大向下距离
        ans  = max(ans, d[u] + d[v] + w);
        d[u] = max(d[u], d[v] + w);
    }
}

/**
 * @brief LCA
 * @details 树上倍增
 * @details 预处理 O(nlogn)，询问 O(logn)
 * @param f[x][k]:表示 x 的 2^k 辈祖先
 * @param d[x]:表示 x 深度
 */

int f[N][LOGN];
int d[N];

void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t   = (int)(log(n) / log(2)) + 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);

            d[v] = d[u] + 1;

            f[v][0] = u;
            for (int j = 1; j <= t; ++j) {
                f[v][j] = f[f[v][j - 1]][j - 1];
            }
        }
    }
}
int lca(int x, int y) {
    if (d[x] > d[y]) {
        swap(x, y);
    }
    int t = (int)(log(n) / log(2)) + 1;

    for (int i = t; i >= 0; --i) {
        if (d[f[y][i]] >= d[x]) {
            y = f[y][i];
        }
    }
    if (x == y) {
        return x;
    }
    for (int i = t; i >= 0; --i) {
        if (f[x][i] != f[y][i]) {
            x = f[x][i];
            y = f[y][i];
```

```
        }
    }
    return f[x][0];
}
```

15.4.4. 树的重心

```
/**
 * @brief 树的重心
 * @details time O(n)
 */
int sz[N];
int now_min = LLONG_MAX;  // 去掉重心后的最大子树大小
int ans;
void dfs(int u, int fa) {
    sz[u] = 1;

    int max_part = 0;  // 表示删掉 x 后分成的最大子树的大小

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == fa) {
            continue;
        }

        dfs(v, u);

        sz[u] += sz[v];

        max_part = max(max_part, sz[v]);
    }
    max_part = max(max_part, n - sz[u]);
    if (max_part < now_min) {
        now_min = max_part;
        ans     = u;
    }
}
```

15.4.5. 树上 RMQ

```
/**
 * @brief 子树权值最大值
 * @note
 *   1. 全树最大权值节点 r 很重要，每个节点朝向节点 r 的子树最大值是 r 的权值
 *   2. 若无 update 操作，则以 r 为根最合适
 *      此时对于每一条边来说
 *      朝向根的子树权值最大值为根
 *      朝向叶子的子树权值最大值正常求
 *   3. 若需要动态维护子树最大值，则可用 dfs 序+线段树
 *      u 的子树对应区间：[dfn[u], dfn[u] + sz[u] - 1]
 */
int a[N];  // 每个节点的权值
int dfn[N];
int dfs_list[N];
int sz[N];
int timestamp;
void dfs(int u, int fa) {
    dfn[u]              = ++timestamp;
    dfs_list[timestamp] = u;
    sz[u]               = 1;
    for (int i = head[u]; ~i; i = edge[i].next) {
```

```
        int v = edge[i].to;
        if (v == fa) {
            continue;
        }
        dfs(v, u);
        sz[u] += sz[v];
    }
}
struct Interval {
    int left, right;
    int node;

    inline int mid(void) const {
        return left + (right - left) / 2;
    }
    inline int len(void) const {
        return right - left + 1;
    }

    inline int left_node(void) const {
        return node * 2;
    }
    inline int right_node(void) const {
        return node * 2 + 1;
    }

    inline int left_len(void) const {
        return mid() - left + 1;
    }
    inline int right_len(void) const {
        return right - mid();
    }
```

```
    inline Interval left_interval(void) const {
        return {left, mid(), node * 2};
    }
    inline Interval right_interval(void) const {
        return {mid() + 1, right, node * 2 + 1};
    }

    inline bool is_leaf(void) const {
        return left == right;
    }
    inline bool is_in(int find_left, int find_right) const {
        return find_left <= left && find_right >= right;
    }
};
struct SegTree {
    vector<int> d;
    vector<int> lazy;
    SegTree(int n) {
        d.assign((n + 1) << 2, 0);
        lazy.assign((n + 1) << 2, 0);
    }
    void push_up(Interval interval) {
        if (!interval.is_leaf()) {
            d[interval.node] = max(d[interval.left_node()], d[interval.right_node()]);
        }
    }
    void build(Interval interval) {
        if (interval.is_leaf()) {
            d[interval.node] = a[dfs_list[interval.left]];
            return;
        }
```

```
        build(interval.left_interval());
        build(interval.right_interval());
        push_up(interval);
    }
    void push_down(Interval interval) {
        if (!interval.is_leaf() && lazy[interval.node]) {
            d[interval.left_node()]     = max(d[interval.left_node()],
lazy[interval.node]);
            d[interval.right_node()]    = max(d[interval.right_node()],
lazy[interval.node]);
            lazy[interval.left_node()]  = max(lazy[interval.left_node()],
lazy[interval.node]);
            lazy[interval.right_node()] = max(lazy[interval.right_node()],
lazy[interval.node]);
            lazy[interval.node]         = 0;
        }
    }
    void update(Interval interval, int find_left, int find_right, int val) {
        if (interval.is_in(find_left, find_right)) {
            d[interval.node]    = max(d[interval.node], val);
            lazy[interval.node] = max(lazy[interval.node], val);
            return;
        }
        push_down(interval);
        int mid = interval.mid();
        if (find_left <= mid) {
            update(interval.left_interval(), find_left, find_right, val);
        }
        if (find_right > mid) {
            update(interval.right_interval(), find_left, find_right, val);
        }
        push_up(interval);
    }
```

```
    int query(Interval interval, int find_left, int find_right) {
        if (find_left > find_right) {
            return 0;
        }
        if (interval.is_in(find_left, find_right)) {
            return d[interval.node];
        }
        push_down(interval);
        int mid = interval.mid();
        int ans = 0;
        if (find_left <= mid) {
            ans = max(ans, query(interval.left_interval(), find_left, find_right));
        }
        if (find_right > mid) {
            ans = max(ans, query(interval.right_interval(), find_left, find_right));
        }
        return ans;
    }
};
SegTree seg;
int n;
void init() {
    cin >> n;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }
    init_edge(n);
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        add_edge(u, v);
    }
```

```
    dfs(1, -1);
    seg.build({1, n, 1});
}
void update_subtree(int u, int val) {
    seg.update({1, n, 1}, dfn[u], dfn[u] + sz[u] - 1, val);
}
int query_nonsubtree(int u) {
    return max(seg.query({1, n, 1}, 1, dfn[u] - 1), seg.query({1, n, 1}, dfn[u] + sz[u], n));
}
```

15.4.6. 树链剖分

15.4.6.1. 重链剖分

```
/**
 * @brief 重链剖分
 * @def 重子节点：表示子结点中子树最大的子节点(若相同则任选其一)
 *      轻子结点：其余子结点
 *      重/轻边：当前结点与对应子结点的连边
 *      重链：首尾相连的重边或落单的结点
 * @property
 *   1. 重链上深度最小结点一定不是重子节点(要么是根，要么是轻子结点)
 *   2. 每向下经过一条轻边时，所在子树大小至少除以 2
 *   3. 可以将树上任何一条路径划分成不超过 O(logn)条连续的链，且每条链上的点深度互不相同
 *   4. 保证划分出的每条链的结点 dfs 序连续，因此可以用线段树、树状数组等结构进行维护树上路径的信息
 * @note
 *   1. 可以用来 O(logn)地求 LCA
 */
int depth[N];
int father[N];
int heavy_son[N];
int sz[N];
int dfn[N];
int rnk[N];
int timestamp;
int top[N];  // 重链上深度最小的结点
void dfs(int u, int fa) {
    father[u] = fa;
    depth[u]  = depth[fa] + 1;
    sz[u]     = 1;
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        if (v == fa) {
            continue;
        }
        dfs(v, u);
        sz[u] += sz[v];
        if (sz[v] > sz[heavy_son[u]]) {
            heavy_son[u] = v;
        }
    }
}
// 建链
void dfs2(int u, int ftop, int fa) {
    top[u]         = ftop;
    dfn[u]         = ++timestamp;
    rnk[timestamp] = u;
    if (heavy_son[u]) {
        dfs2(heavy_son[u], ftop, u);
    }
    for (int i = head[u]; ~i; edge[i].next) {
        int v = edge[i].to;
```

```
        if (v != heavy_son[u] && v != fa) {
            dfs2(v, v, u);
        }
    }
}
```

```
// 用重链剖分求两点路径权值之和
int a[N];
int presum[N];  // 预处理链上前缀和
void dfs_presum(int u, int fa) {
    presum[u] += a[u];
    for (int i = head[u]; ~i; edge[i].next) {
        int v = edge[i].to;
        if (v == fa) {
            continue;
        }
        if (v == heavy_son[u]) {
            presum[v] += presum[u];
        }
        dfs_presum(v, u);
    }
}
int path_sum(int u, int v) {
    int ans = 0;
    while (top[u] != top[v]) {
        if (depth[top[u]] < depth[top[v]]) {
            swap(u, v);
        }
        ans += presum[u];
        u = father[top[u]];
    }
    return ans;
}
```

```
// 求 lca
int lca(int u, int v) {
    while (top[u] != top[v]) {
        if (depth[top[u]] > depth[top[v]]) {
            u = father[top[u]];
        }
        else {
            v = father[top[v]];
        }
    }
    return depth[u] > depth[v] ? v : u;
}
```

15.5. 生成树

15.5.1. 最小生成树

15.5.1.1. Kruskal 算法

```
/**
 * @brief Kruskal 算法
 * @details time O(m log m)
 */
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
```

```
    }

    size_t find_root(size_t x) {
        return parent[x] == x ? x : parent[x] = find_root(parent[x]);
    }
    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};


struct Temp_edge {
    int u, v, w;
};
bool operator<(const Temp_edge& a, const Temp_edge& b) {
    return a.w < b.w;
}
void kruskal(vector<Temp_edge>& lists) {
    int n = lists.size();
    sort(lists.begin(), lists.end());


    DSU dsu(n);
    for (auto [u, v, w] : lists) {
        int x = dsu.find_root(u);
        int y = dsu.find_root(v);
        if (x == y) {
            continue;
        }
        dsu.unite(x, y);


        // add_edge(u, v, w);
    }
}
```

15.5.1.2. Prim 算法

```
/**
 * @brief Prim 算法
 * @details O(m log n)
 */
const int N = 1e5 + 5;
int dis[N];  // 到当前生成树的最小距离
// pii=<dis,u>
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
vector<bool> visited;
int prim(int root) {
    visited.resize(n, false);
    memset(dis, 0x3f, sizeof(dis));
    dis[root] = 0;
    q.push({0, root});
    int cnt = 0;
    while (!q.empty()) {

        auto [d, u] = q.top();
```

```
        q.pop();

        if (visited[u]) {
            continue;
        }
        visited[u] = true;

        ++cnt;

        // do_something(father,u,d);

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;

            if (w < dis[v]) {
                dis[v] = w;
                q.push({w, v});
            }
        }

        if (cnt >= n) {
            break;
        }
    }
    if (cnt < n) {
        // 不连通
        return -1;
    }
    return 0;
```

```
}
```

15.6. 二分图

15.6.1. 染色判定法

```
/**
 * @brief 二分图的染色判定法
 */
int color[N];
bool dfs_color(int u, int c) {
    color[u] = c;
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (color[v] == 0) {
            if (!dfs_color(v, 3 - c)) {
                return false;
            }
        }
        else if (color[v] == c) {
            return false;
        }
    }
    return true;
}
bool judge() {
    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) {
            if (!dfs_color(i, 1)) {
```

```
                return false;
            }
        }
    }
    return true;
}
```

15.6.2. 最大匹配数(匈牙利算法)

```
/**
 * @brief 二分图的最大匹配
 * @brief 匈牙利算法
 * @details time O(n m)
 * @return 最大匹配数
 */
int match[N];
int visited[N];
bool dfs(int u) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        if (!visited[v]) {
            visited[v] = true;
            if (!match[v] || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}
```

```
int Hungarian() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            ans++;
        }
    }
    return ans;
}
```

15.7. 网络流

15.7.1. 最大流

15.7.1.1. Edmonds-Karp 算法

```
/**
 * @name Edmonds-Karp 算法
 * @details 时间复杂度 O(V * E^2)
 */
struct Edge {
    int next;
    int to;
    int weight;  // capacity
};
vector<Edge> edge;

vector<int> head(N, -1);
```

```
void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v, int w) {
    // 正图
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;
    // 反图
    edge.push_back({head[v], u, 0});
    head[v] = edge.size() - 1;
}
// pre[v] 记录到达 v 的边的下标
int pre[N];
// minflow[v] 是 s 到 v 的最小剩余容量
int minflow[N];

bool bfs(int s, int t) {
    fill(pre, pre + N, -1);
    memset(minflow, 0, sizeof(minflow));
    queue<int> q;
    q.push(s);
    minflow[s] = INF;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            // 未访问或有剩余容量
            if (pre[v] == -1 && w > 0) {
                pre[v]     = i;
                minflow[v] = min(minflow[u], w);
                q.push(v);
            }
        }
    }
    return pre[t] != -1;
}
int max_flow(int s, int t) {
    int flow = 0;
    while (bfs(s, t)) {  // bfs 找路
        // 从 t 开始遍历沿着反图走到 s
        for (int v = t; v != s;) {
            int i = pre[v];
            edge[i].weight -= minflow[t];
            edge[i ^ 1].weight += minflow[t];
            v = edge[i ^ 1].to;
        }
        flow += minflow[t];
    }
    return flow;
}
```

15.7.1.2. Dinic 算法

```
/**
 * @name Dinic 算法
 * @details 时间复杂度 O(V^2 * E)
 */
struct Edge {
```

```
    int next;
    int to;
    int weight;
};
vector<Edge> edge;
vector<int> head(N, -1);
void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v, int w) {
    // 正图
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;
    // 反图
    edge.push_back({head[v], u, 0});
    head[v] = edge.size() - 1;
}

vector<int> level(N, -1);  // 分层图
vector<int> cur(N, -1);    // 当前弧优化，cur[u]表示遍历到哪条边
// bfs 找路，建立分层图，让 dfs 只允许走最短路
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);

    queue<int> q;
    q.push(s);
    level[s] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            // 如果没有经过 v 或有剩余容量
            if (level[v] == -1 && w > 0) {
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }
    return level[t] != -1;
}
// 按分层图找 u 到 t 的增广路
int dfs(int u, int t, int flow) {
    if (u == t) {
        return flow;  // 找到增广路即返回
    }
    for (int& i = cur[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        // 只能走到下一层且有剩余容量的点
        if (level[v] == level[u] + 1 && w > 0) {
            int d = dfs(v, t, min(flow, w));
            // 如果之后的 v 到 t 有最大流量 d
            if (d > 0) {
                edge[i].weight -= d;
                edge[i ^ 1].weight += d;  // 反向边
                return d;
            }
        }
    }
```

```
    return 0;
}
int max_flow(int s, int t) {
    int ans = 0;
    while (bfs(s, t)) {
        copy(head.begin(), head.end(), cur.begin());
        int f = 0;
        while ((f = dfs(s, t, INF)) > 0) {
            ans += f;
        }
    }
    return ans;
}
```

15.8. 连通性

15.8.1. 强连通分量 SCC

15.8.1.1. Tarjan 算法

```
/**
 * @name Tarjan 算法
 * @details O(V+E)
 */
struct Edge {
    int next;
    int to;
    int weight;
};
vector<Edge> edge;
vector<int> head(N, -1);

int dfn[N];     // dfs 搜索时，u 被搜索到的 timestamp。从根节点开始的一个路径上 dfn[u]单调递增
int low[N];     // 从 u 出发，dfs 子树中，能追溯到的最早的仍在栈里的节点的 dfn。从根节点开始的一个路径上 low[u]单调不增
int timestamp;  // 全局时间戳

int SCC_count;    // SCC 个数
int SCC_id[N];    // 节点 i 所在的 SCC 编号
int SCC_size[N];  // 编号为 i 的 SCC 的大小

bool in_stack[N];  // 节点 u 是否在栈中
stack<int> s;      // 栈

void tarjan(int u) {
    low[u] = dfn[u] = ++timestamp;

    s.push(u);
    in_stack[u] = true;
    // 遍历一次 u 树
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        // 如果 v 没有被搜索过
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        }
        // 如果 v 被搜索过并且在栈中，则说明找到了返祖边
        else if (in_stack[v]) {
            low[u] = min(low[u], dfn[v]);
        }
```

```
    }
    // 找到 SCC 的根节点
    if (dfn[u] == low[u]) {
        ++SCC_count;
        int v;
        // 弹出我和我的子树(即找到了一组 SCC)
        do {
            v = s.top();
            s.pop();
            SCC_id[v] = SCC_count;
            ++SCC_size[SCC_count];
            in_stack[v] = false;
        } while (u != v);
    }
}
void solve() {
    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) {
            tarjan(i);
        }
    }
}
```

15.8.1.2. Kosaraju 算法

```
/**
 * @brief Kosaraju 算法
 * @details O(V+E)
 */
vector<int> edge[N];  // 正向图
vector<bool> vis;
vector<int> edge2[N];  // 反向图
vector<bool> vis2;

vector<int> post_order;  // 后序遍历节点表

int SCC_count;
int SCC_id[N];
int SCC_size[N];

// 正图上后序遍历，得到拓扑序
void dfs1(int u) {
    vis[u] = true;
    for (auto v : edge[u]) {
        if (!vis[v]) {
            dfs1(v);
        }
    }
    post_order.push_back(u);
}
// 反图上扩散
void dfs2(int u) {
    SCC_id[u] = SCC_count;
    ++SCC_size[SCC_count];
    for (auto v : edge2[u]) {
        if (!SCC_id[v]) {
            dfs2(v);
        }
    }
}

void kosaraju() {
    // 第一次 dfs 进行后序遍历
    for (int i = 1; i <= n; ++i) {
```

```
        if (!vis[i]) {
            dfs1(i);
        }
    }
    // 对后序遍历的表进行反向遍历
    for (auto it = post_order.rbegin(); it != post_order.rend(); ++it) {
        int u = *it;
        // 如果还没开始标 SCC 的序号
        if (!SCC_id[u]) {
            ++SCC_count;
            dfs2(u);
        }
    }
}
```

15.8.2. 双连通分量 BCC

15.8.2.1. e-BCC

```
/**
 * @brief tarjan 算法
 * @details 无向图，边-双连通即为任意去掉一条边仍然互相连通
 * @details O(E+V)
 */
struct Edge {
    int next;
    int to;
};
vector<Edge> edge;
vector<int> head(N, -1);

void add_edge(int u, int v) {
    edge.push_back({head[u], v});
    head[u] = edge.size() - 1;
    edge.push_back({head[v], u});
    head[v] = edge.size() - 1;
}

int dfn[N];
int low[N];
int timestamp;

bool is_bridge[N];  // 边 i 是否为桥

int BCC_count;    // e-BCC 个数
int BCC_id[N];    // 节点 u 所在 BCC 编号
int BCC_size[N];  // 编号为 i 的 BCC 大小

void tarjan(int u, int in_edge) {  // 节点 u，入节点 u 的边
    low[u] = dfn[u] = ++timestamp;
    for (int i = head[u]; ~i; i = edge[i].next) {
        // 不能回去
        if (i == (in_edge ^ 1)) {
            continue;
        }

        int v = edge[i].to;

        // v 没有被搜索过->树边
        if (!dfn[v]) {
            tarjan(v, i);
            low[u] = min(low[u], low[v]);
            if (low[v] > dfn[u]) {
```

```
                is_bridge[i] = is_bridge[i ^ 1] = true;
            }
        }
        // v 被搜索过->回边
        else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}

void dfs_BCC(int u) {
    BCC_id[u] = BCC_count;
    ++BCC_size[BCC_count];

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        // 是桥或者已标号
        if (is_bridge[i] || BCC_id[v]) {
            continue;
        }
        dfs_BCC(v);
    }
}
void solve() {
    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) {
            tarjan(i, -1);
        }
    }
    for (int i = 1; i <= n; ++i) {
        if (!BCC_id[i]) {
            ++BCC_count;
            dfs_BCC(i);
        }
    }
}
```

15.8.2.2. v-BCC

```
/**
 * @note 两个 v-BCC 最多有一个公共点，其为割点
 * @note v-BCC 在 dfs 搜索树中 dfn 最小的点 一定是割点或树根
 *       1. 当这个点为割点时，一定是点双连通分量的根
 *       2. 当这个点为树根时，
 *          a. 有两个以上的子树，那他一定是割点
 *          b. 只有一个子树，那他是一个点双连通分量的根
 *          c. 没有子树，那他自己成为一个点双连通分量
 * @details O(V+E)
 */
struct Edge {
    int next;
    int to;
};
vector<Edge> edge;
vector<int> head(N, -1);
void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v) {
    edge.push_back({head[u], v});
    head[u] = edge.size() - 1;
    edge.push_back({head[v], u});
```

```
    head[v] = edge.size() - 1;
}

int dfn[N];
int low[N];
int timestamp;

bool is_cut[N];

int root;  // 此时 dfs 搜索树是以谁为 root
stack<int> s;

int BCC_count;       // v-BCC 个数
vector<int> BCC[N];  // BCC[i]表示第 i 个 v-BCC，注：有些割点是重复的

void tarjan(int u) {
    dfn[u] = low[u] = ++timestamp;
    s.push(u);

    int child_num = 0;  // 记录满足 low[v]>=dfn[u]的子节点个数
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
            // 判据：无法回溯到更早的地方，则 u 一定是一个割点或根节点
            if (low[v] >= dfn[u]) {
                // 如果不是树根，一定为割点；如果有两个以上的子树，则一定为割点
                if (++child_num > 1 || u != root) {
                    is_cut[u] = true;
                }
                // 说明发现 v-BCC
                ++BCC_count;
                int x;
                do {
                    x = s.top();
                    s.pop();
                    BCC[BCC_count].push_back(x);
                } while (x != v);
                BCC[BCC_count].push_back(u);
            }
        }
        else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
void solve() {
    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) {
            if (~head[i]) {
                while (!s.empty()) {
                    s.pop();
                }
                root = i;
                tarjan(i);
            }
            else {
                dfn[i] = low[i] = ++timestamp;
                ++BCC_count;
                BCC[BCC_count].push_back(i);
            }
```

```
        }
    }
}
```

16. 字符串

16.1. hash

```
const size_t base = 131;
const size_t mod1 = 212370440130137957, mod2 = 1e9 + 7;
size_t get_hash1(const string& s) {
    size_t ans = 0;
    for (const auto& c : s) {
        ans = (ans * base + c) % mod1;
    }
    return ans;
}
size_t get_hash2(const string& s) {
    size_t ans = 0;
    for (const auto& c : s) {
        ans = (ans * base + c) % mod2;
    }
    return ans;
}
bool is_equal(const string& s1, const string& s2) {
    return get_hash1(s1) == get_hash1(s2) && get_hash2(s1) == get_hash2(s2);
}
```

16.2. KMP 模式匹配

```
string s;
string k;
```

```
vector<int> kmp;
void set_kmp(void) {
    kmp[0] = 0;
    for (int i = 1; i <= k.size(); ++i) {
        int j = kmp[i - 1];
        while (j > 0 && k[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (k[i] == k[j]) {
            ++j;
        }
        kmp[i] = j;
    }
}
int main() {
    cin >> s >> k;
    kmp.resize(k.size());
    set_kmp();
    for (int i = 0, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (s[i] == k[j]) {
            ++j;
        }
        if (j == k.size()) {
            cout << i - k.size() + 2 << endl;
            j = kmp[j - 1];
        }
    }
    for (int i = 0; i < kmp.size(); ++i) {
        cout << kmp[i] << " ";
    }
    return 0;
}
```

16.3. Trie 字典树

```
struct trie {
    int next[100000][26];
    int cnt;
    bool exist[100000];

    void insert(const char* s, int len) {
        int ptr = 0;
        for (int i = 0; i < len; ++i) {
            int c = s[i] - 'a';
            if (!next[ptr][c]) {
                ++cnt;
                next[ptr][c] = cnt;
            }
            ptr = next[ptr][c];
        }
        exist[ptr] = true;
    }
    bool find(const char* s, int len) {
        int ptr = 0;
        for (int i = 0; i < len; ++i) {
            int c = s[i] - 'a';
            if (!next[ptr][c]) {
                return false;
            }
            ptr = next[ptr][c];
```

```
        }
        return exist[ptr];
    }
};
```

16.4. AC 自动机

```
struct Trie {
    int son[26];
    int fail;
    int exist;
    bool visited;
} t[N];

// 节点编号
int cnt;

// trie 普通构建
void insert(const char* s) {
    int u = 0;
    for (int i = 0; s[i]; ++i) {
        int c  = s[i] - 'a';
        int& v = t[u].son[c];
        if (!v) {
            v = ++cnt;
        }
        u = v;
    }
    ++t[u].exist;
}

// AC 自动机构建
```

```
void build() {
    queue<int> q;
    for (int c = 0; c < 26; ++c) {
        if (t[0].son[c]) {
            q.push(t[0].son[c]);
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int c = 0; c < 26; ++c) {
            int& v = t[u].son[c];
            if (v) {
                t[v].fail = t[t[u].fail].son[c];
                q.push(v);
            }
            else {
                v = t[t[u].fail].son[c];
            }
        }
    }
}

// 一次匹配，不需要 visited 标识
int query_once(const char* s) {
    int u   = 0;
    int ret = 0;  // 匹配数
    for (int i = 0; s[i]; ++i) {
        int c = s[i] - 'a';

        u = t[u].son[c];
        for (int j = u; j && t[j].exist != -1; j = t[j].fail) {
```

```
            ret += t[j].exist;
            t[j].exist = -1;  // exist=-1 指已经询问过了，但这种方法只能询问一次，若想多次询问则需要构建 visited 状态
        }
    }
    return ret;
}
// 多次匹配
int query_multi(const char* s) {
    // 清空状态
    for (int i = 0; i <= cnt; ++i) {
        t[i].visited = false;
    }
    // 再匹配
    int u   = 0;
    int ret = 0;  // 匹配数
    for (int i = 0; s[i]; ++i) {
        int c = s[i] - 'a';

        u = t[u].son[c];
        for (int j = u; j && !t[j].visited; j = t[j].fail) {
            ret += t[j].exist;
            t[j].visited = true;
        }
    }
    return ret;
}
```

16.5. SA

```
string s;
int sa[N << 1];
int rk[N << 1];

// O(n^2 log n)
void get_sa1(void) {
    iota(sa, sa + s.size(), 0);
    sort(sa, sa + s.size(), [&](int i, int j) {
        return s.substr(i) < s.substr(j);
    });
}
void get_rk_from_sa(void) {
    for (int i = 0; i < s.size(); ++i) {
        rk[sa[i]] = i;
    }
}
// O(n log^2 n)
int oldrk[N << 1];
void get_sa2(void) {
    // 第一层排序
    for (int i = 0; i < s.size(); ++i) {
        sa[i] = i;
        rk[i] = s[i];
    }
    // 二层及以上排序
    for (int w = 1; w < s.size(); w <<= 1) {
        sort(sa, sa + s.size(), [&](int i, int j) {
            // 这里需要两倍空间，否则就得老老实实防止越界
            return rk[i] == rk[j] ? rk[i + w] < rk[j + w] : rk[i] < rk[j];
        });
        for (int i = 0; i < s.size(); ++i) {
            oldrk[i] = rk[i];
        }
        // 双指针去重
```

```
        rk[sa[0]] = 0;
        for (int i = 1, j = 0; i < s.size(); ++i) {
            if (oldrk[sa[i]] == oldrk[sa[i - 1]] && oldrk[sa[i] + w] == oldrk[sa[i - 1] +
w]) {
                rk[sa[i]] = j;
            }
            else {
                rk[sa[i]] = ++j;
            }
        }
    }
}
```

16.6. SAM

```
struct state {
    int len;
    int link;
    map<char, int> next;
};
const int N = 1000000;
state st[N * 2];
int sz, last;
void sam_init() {
    st[0].len  = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}
void sam_extend(char c) {
    int cur     = sz++;
    st[cur].len = st[last].len + 1;
    int p       = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p             = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    }
    else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        }
        else {
            int clone      = sz++;
            st[clone].len  = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p             = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
void sam_build(string s) {
    sam_init();
    for (auto x : s) {
        sam_extend(x);
```

```
    }
}
```

17. 随机化

17.1. 随机函数

17.1.1. 生成函数

```cpp
/**
 * @brief 随机数生成函数
 */
// 基于硬件的均匀分布随机数生成器
static random_device rd;
// 32 位梅森缠绕器(@ret u32)
// 可替换为 64 位 mt19937_64
static mt19937 gen(rd());
// uniform_int_distribution 为整数，左闭右闭[]
// 浮点数用 uniform_real_distribution，左闭右开[)
static uniform_int_distribution<int> dist(1, 100);
int get_rand() {
    return dist(gen);
}
```

17.1.2. 随机打乱

```cpp
/**
 * @brief 随机打乱
 */
void algo_shffle(vector<int>& v) {
    shuffle(v.begin(), v.end(), gen);  // 此处 gen 可换为一个随机数发生函数
```

```
}
```

17.1.3. 随机挑选

```
/**
 * @brief 随机挑选
 */
vector<int> algo_sample(const vector<int>& v, int n) {
    vector<int> ret;
    sample(v.begin(), v.end(), ret.end(), n, gen);
    return ret;
}
```

17.2. 爬山与退火

18. 离线算法

18.1. 整体二分

18.1.1. 区间第 k 小

```
/**
 * @brief 整体二分+树状数组
 * @details 静态区间第 k 小
 * @details 给定长度为 n 的序列 a，m 次询问区间 [l,r] 内第 k 小值。
            输入：第一行 n,m；第二行 a_1,...,a_n；接下来 m 行每行 l,r,k。
            数据：1 ≤ n,m ≤ 2e5，0 ≤ a_i ≤ 1e9，1 ≤ l ≤ r ≤ n，1 ≤ k ≤ r-l+1。
 */
INCLUDE...
// #define int long long

const int N = 2e5 + 5;

int n;
//==================BIT==================
int lowbit(int x) {
    return x & (-x);
}
// int b[N];  // 原数组第 i 个位置的值，是否小于等于当前的 mid
int c[N];  // 树状数组节点，c[i] 管理 (i - lowbit(i), i] 这段区间
// 单点更新
void add(int pos, int val) {
    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
```

```
    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {
    return presum(right) - presum(left - 1);
}

//=================整体二分====================

int a[N];             // 原数据->按照原序的离散化的数据
pair<int, int> b[N];  // 排序离散后的数据<value,idx>
int toRaw[N];         // rank 下标对应的值

int m;

int ans[N];

struct Event {
    enum EventType {
        ADD,
        QUERY
    } event_type;

    int l, r, k;  // query 的内容
    int idx;      // 在对应数组中的 idx
};
vector<Event> event;
vector<Event> leftE, rightE;

// el,er: Query 的 idx
// L,R: 值域
void solve(int el, int er, int L, int R) {
    if (el > er) {
        return;
    }

    if (L == R) {
        for (int i = el; i <= er; ++i) {
            if (event[i].event_type == Event::QUERY) {
                ans[event[i].idx] = L;
            }
        }
        return;
    }

    int MID = (L + R) >> 1;

    leftE.clear(), rightE.clear();
    for (int i = el; i <= er; ++i) {
        if (event[i].event_type == Event::ADD) {
            int idx = event[i].idx;
            if (a[idx] <= MID) {
                add(idx, 1);
                leftE.push_back(event[i]);
            }
            else {
```

```
                rightE.push_back(event[i]);
            }
        }
    }

    for (int i = el; i <= er; ++i) {
        if (event[i].event_type == Event::QUERY) {
            auto& e = event[i];
            int cnt = query(e.l, e.r);
            if (cnt >= e.k) {
                leftE.push_back(event[i]);
            }
            else {
                e.k -= cnt;
                rightE.push_back(event[i]);
            }
        }
    }
    // 只需要清空左半边的 event 即可，因为右半边的插入事件本来就不会被 add 进入
    for (const auto& e : leftE) {
        if (e.event_type == Event::ADD) {
            int idx = e.idx;
            if (a[idx] <= MID) {
                add(idx, -1);
            }
        }
    }

    int cnt = 0;
    for (const auto& x : leftE) {
        event[el + cnt] = x;
        cnt++;
    }
    int mid = el + cnt;
    for (const auto& x : rightE) {
        event[el + cnt] = x;
        cnt++;
    }
    solve(el, mid - 1, L, MID);
    solve(mid, er, MID + 1, R);
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;

    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        b[i] = {a[i], i};
    }

    //=================离散化=============
    sort(b + 1, b + 1 + n);
    int rank       = 0;
    int last_value = -1;
    for (int i = 1; i <= n; ++i) {
        auto [v, idx] = b[i];
        if (v != last_value) {
            ++rank;
            last_value = v;
        }
        a[idx]      = rank;
        toRaw[rank] = v;
```

```
    }
    //=================事件化==============
    for (int i = 1; i <= n; ++i) {
        Event e;
        e.event_type = Event::ADD;
        e.idx        = i;
        event.push_back(e);
    }
    for (int i = 1; i <= m; ++i) {
        Event e;
        e.event_type = Event::QUERY;
        cin >> e.l >> e.r >> e.k;
        e.idx = i;
        event.push_back(e);
    }

    solve(0, event.size() - 1, 1, rank);

    for (int i = 1; i <= m; ++i) {
        cout << toRaw[ans[i]] << endl;
    }
    return 0;
}
```

18.1.2. 单点修改区间第 k 小

```
/**
 * @brief 整体二分+树状数组
 * @details 单点修改静态区间第 k 小
 * @details 给定长度为 n 的序列 a，m 次操作：Q l r k 查询区间 [l,r] 内第 k 小值；C x y 将 a_x 改为 y。
            输入：第一行 n,m；第二行 a_1,...,a_n；接下来 m 行每行一个操作 Q l r k 或 C x y。
            数据：1 ≤ n,m ≤ 1e5，1 ≤ l ≤ r ≤ n，1 ≤ k ≤ r-l+1，1 ≤ x ≤ n，0 ≤ a_i,y ≤ 1e9。
 * @note 也可使用离散化，但是不离散最多 30 层，不离散 18 层，只差常数级
 */
INCLUDE...
using namespace std;
// #define int long long

const int N = 2e5 + 5;

int n;
//==================BIT==================
int lowbit(int x) {
    return x & (-x);
}
// int b[N];  // 原数组第 i 个位置的值，是否小于等于当前的 mid
int c[N];  // 树状数组节点，c[i] 管理 (i - lowbit(i), i] 这段区间
// 单点更新
void add(int pos, int val) {
    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {
```

```
    return presum(right) - presum(left - 1);
}

//=================整体二分====================

int a[N];  // 原数据

int m;

int ans[N];

struct Event {
    enum EventType {
        ADD,
        DELETE,
        QUERY
    } event_type;

    int l, r, k;  // query 的内容
    int idx;      // 在对应数组中的 idx
};
vector<Event> event;
vector<Event> leftE, rightE;

// el,er: Query 的 idx
// L,R: 值域
void solve(int el, int er, int L, int R) {
    if (el > er) {
        return;
    }

    if (L == R) {
        for (int i = el; i <= er; ++i) {
            if (event[i].event_type == Event::QUERY) {
                ans[event[i].idx] = L;
            }
        }
        return;
    }

    int MID = (L + R) >> 1;

    leftE.clear(), rightE.clear();
    for (int i = el; i <= er; ++i) {
        if (event[i].event_type == Event::QUERY) {
            auto& e = event[i];
            int cnt = query(e.l, e.r);
            if (cnt >= e.k) {
                leftE.push_back(event[i]);
            }
            else {
                e.k -= cnt;
                rightE.push_back(event[i]);
            }
        }
        else {
            auto& e = event[i];
            if (e.k <= MID) {
                add(e.idx, e.event_type == Event::ADD ? 1 : -1);
                leftE.push_back(e);
            }
            else {
                rightE.push_back(e);
            }
```

```
        }
    }

    // 只需要清空左半边的 event 即可，因为右半边的事件本来就不会被 add 进入
    for (const auto& e : leftE) {
        if (e.event_type == Event::ADD || e.event_type == Event::DELETE) {
            int idx = e.idx;
            if (e.k <= MID) {
                add(idx, e.event_type == Event::ADD ? -1 : 1);
            }
        }
    }

    int cnt = 0;
    for (const auto& x : leftE) {
        event[el + cnt] = x;
        cnt++;
    }
    int mid = el + cnt;
    for (const auto& x : rightE) {
        event[el + cnt] = x;
        cnt++;
    }
    solve(el, mid - 1, L, MID);
    solve(mid, er, MID + 1, R);
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;
```

```
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }

    //=================事件化==============
    for (int i = 1; i <= n; ++i) {
        Event e;
        e.event_type = Event::ADD;
        e.idx        = i;
        e.k          = a[i];
        event.push_back(e);
    }
    for (int i = 1; i <= m; ++i) {
        char c;
        cin >> c;
        if (c == 'C') {
            Event e;
            e.event_type = Event::DELETE;
            cin >> e.idx;
            e.k = a[e.idx];
            event.push_back(e);
            e.event_type = Event::ADD;
            cin >> e.k;
            a[e.idx] = e.k;

            event.push_back(e);
        }
        else {
            Event e;
            e.event_type = Event::QUERY;
            cin >> e.l >> e.r >> e.k;
            e.idx = i;
```

```
            event.push_back(e);
        }
    }

    fill(ans + 1, ans + m + 1, -1);

    solve(0, event.size() - 1, 0, 1e9 + 1);

    for (int i = 1; i <= m; ++i) {
        if (ans[i] != -1) {
            cout << ans[i] << endl;
        }
    }
    return 0;
}
```

18.2. 莫队算法

18.2.1. 普通莫队

```
/**
 * @brief 普通莫队
 */
int n, m;  // n 是点数，m 是询问数
int block_size;

void init() {
    block_size = sqrt(n);  // 有时候 n/sqrt(m)会更快
}
int block(int x) {
    return x / block_size;
}
```

```
}
struct Query {
    int l, r;
    int id;

    int len() const {
        return (r - l + 1);
    }

    // 奇偶化排序
    bool operator<(const Query& _) const {
        if (block(l) != block(_.l)) {
            return l < _.l;
        }
        return (block(l) & 1) ? r < _.r : r > _.r;
    }
} q[N];

int ans[N];

int c[N];  // 数据

void add(int x) {
}
void del(int x) {
}

void solve() {
    sort(q + 1, q + m + 1);

    int curL = 1, curR = 0;  // 左闭右闭
    for (int i = 1; i <= m; ++i) {
```

```
        // 注意 add 是先自增自减，del 是后自增自减
        // 最好不要改变这里四个 while 的顺序
        // 可用的 while 顺序：先扩大后缩小（当然不只有这四种）
        while (curL > q[i].l)
            add(--curL);
        while (curR < q[i].r)
            add(++curR);
        while (curL < q[i].l)
            del(curL++);
        while (curR > q[i].r)
            del(curR--);

        ans[q[i].id] = ;
    }
}
```

18.2.2. 单点修改的莫队

```
/**
 * @brief 单点修改的莫队
 * @note 只需要增加一个维度—修改维度
 * @note operator<的处理为：再在后面加一个 return t<_.t
 * @note 新加的两个 while 应放到其他四个 while 前面或后面
 */
```